Desplegar Flutter Web en Vercel (CLI)

🎯 **Objetivo**

El objetivo es **publicar nuestra aplicación Flutter Web en internet** para que cualquier persona pueda acceder mediante una URL.

Para ello utilizaremos **Vercel**, una plataforma en la nube que permite alojar aplicaciones web de forma sencilla y gratuita.

📌 **¿Qué es Vercel?**
Vercel es un servicio de hosting que permite subir proyectos web y obtener una dirección pública del tipo:

```
https://mi-proyecto.vercel.app
```

Con esto conseguimos que nuestra app deje de estar solo en nuestro ordenador y esté disponible online.

✅ **0) Requisitos previos**

- **Flutter instalado y proyecto funcionando**

  👉 Confirmamos que la app funciona antes de publicarla (`flutter doctor`).
- **Node.js instalado**
  - 📌 *Node.js* es el entorno que permite ejecutar herramientas en la terminal (JavaScript fuera del navegador).
  - Nos permite poder usar herramientas como Vercel desde consola.
- **npm disponible (se instala junto con Node)**
  - 📌 *npm* sirve para instalar herramientas desde la terminal.
    - **npm** instala paquetes.
    - **npx** ejecuta paquetes sin instalarlos globalmente.
- **Cuenta en Vercel**
  - 🌐 Crear cuenta en: `https://vercel.com`

🚀 **1) Generar el build web (release)**

📍 Ejecutar en la **raíz del proyecto** (terminal de VS Code o cualquier consola):

```
flutter build web --release
```

📌 Se crea la carpeta: `build/web`

👉 Esto genera los archivos web optimizados (HTML, CSS y JS) y tener la app lista para subir a internet.

🌐 2) Instalar Vercel CLI (solo la primera vez)

📍 Se puede ejecutar en **la terminal de VS Code o cualquier otra consola** (PowerShell, CMD, Terminal, Bash):

```
npm i -g vercel
```

👉 Esto instala el comando vercel en tu ordenador y conseguimos poder desplegar aplicaciones desde la terminal.

📂 3) Entrar en la carpeta del build

Ejecutar desde la raíz del proyecto: `cd build/web`

Nos situamos en la carpeta exacta que vamos a publicar para subir únicamente los archivos necesarios.

☁️ 4) Primer despliegue (preview)

📍 Ejecutar dentro de build/web: `vercel`

👉 Sube la aplicación y crea un despliegue de prueba. Conseguimos una URL temporal para comprobar que todo funciona.

🏁 5) Publicar en producción

📍Ejecutar dentro de build/web: `vercel --prod`

👉 Esto lo que hace es Publica la versión oficial en producción y conseguimos que la URL definitiva quede actualizada (sobrescribe la anterior).

🔄 6) Actualizar la app cuando hagas cambios

Cada vez que modifiques el proyecto debes:

- Generar nuevo build: `flutter build web --release`
- Volver a publicar: `cd build/web` y ejecutar `vercel --prod`

👉 Esto reconstruye vuelve a publicar la versión nueva. Los cambios se reflejen en la web pública.

🚫 7) Arreglo 404 al recargar rutas internas (SPA)

Si al recargar una ruta (ej: /detalle/1) aparece error 404:

1. Crear dentro de build/web un archivo llamado **vercel.json** con este contenido:

```
{
  "rewrites": [{ "source": "/(.*)",
  "destination": "/index.html" }]
}
```

2. Volver a publicar: `vercel --prod`

👉 Esto redirige cualquier ruta a `index.html`. Conseguimos que Flutter Web funcione correctamente al refrescar páginas.

✅ Resumen ultra rápido

1️⃣ `flutter build web --release`
2️⃣ `cd build/web`
3️⃣ `vercel --prod`

🎯 Build → Entrar en carpeta → Publicar → Repetir cuando haya cambios